

Elaborazione 3

UC1.3: Consulta Metriche

Analisi

Vogliamo realizzare un sistema che permetta di estrarre ed eventualmente salvare una serie di metriche relative ad uno o più targets. Le metriche saranno estrapolate dalla cronologia a partire dagli eventi salvati all'interno di un periodo di tempo definito dall'utente. Nel caso di gruppi di dispositivi saranno calcolate delle metriche aggregate. Le metriche selezionate dall'utente saranno mostrate in un report che potrà, opzionalmente, essere salvato per consultazioni successive.

Modello di Dominio

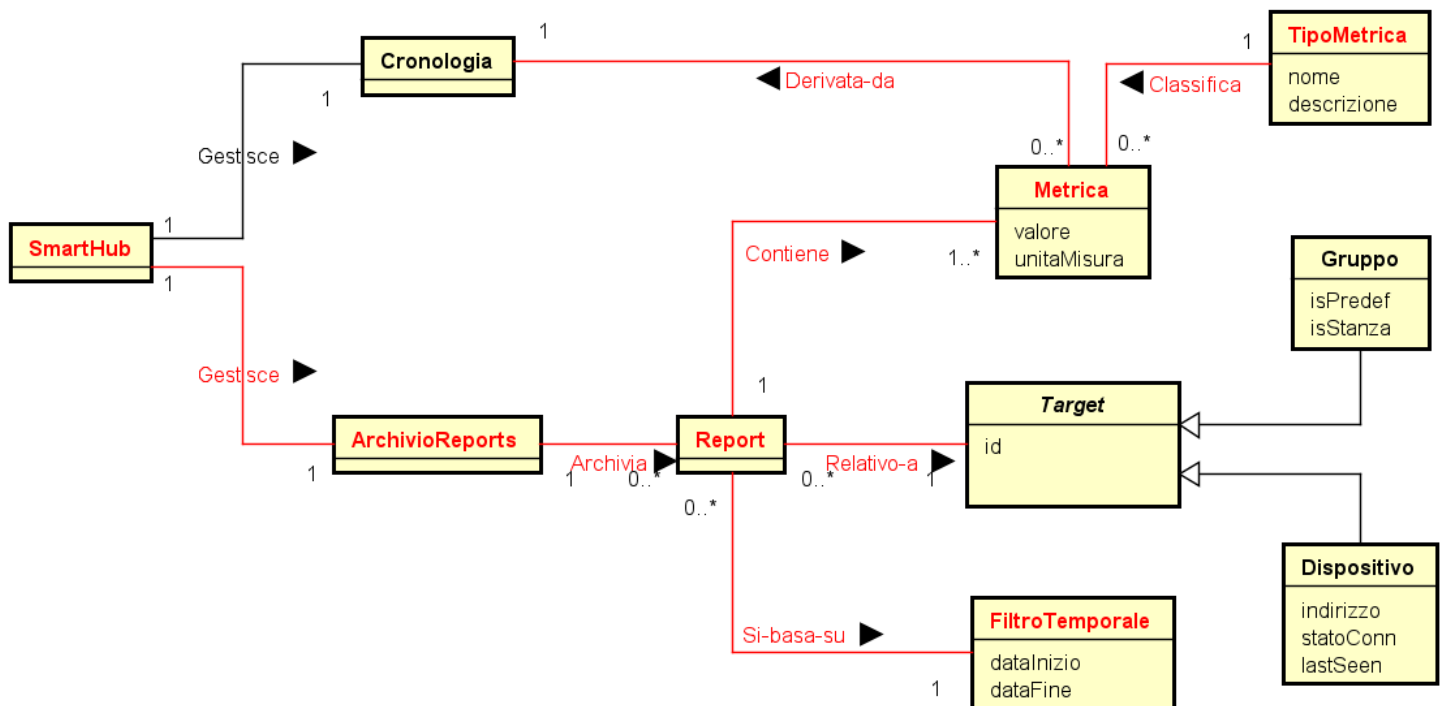
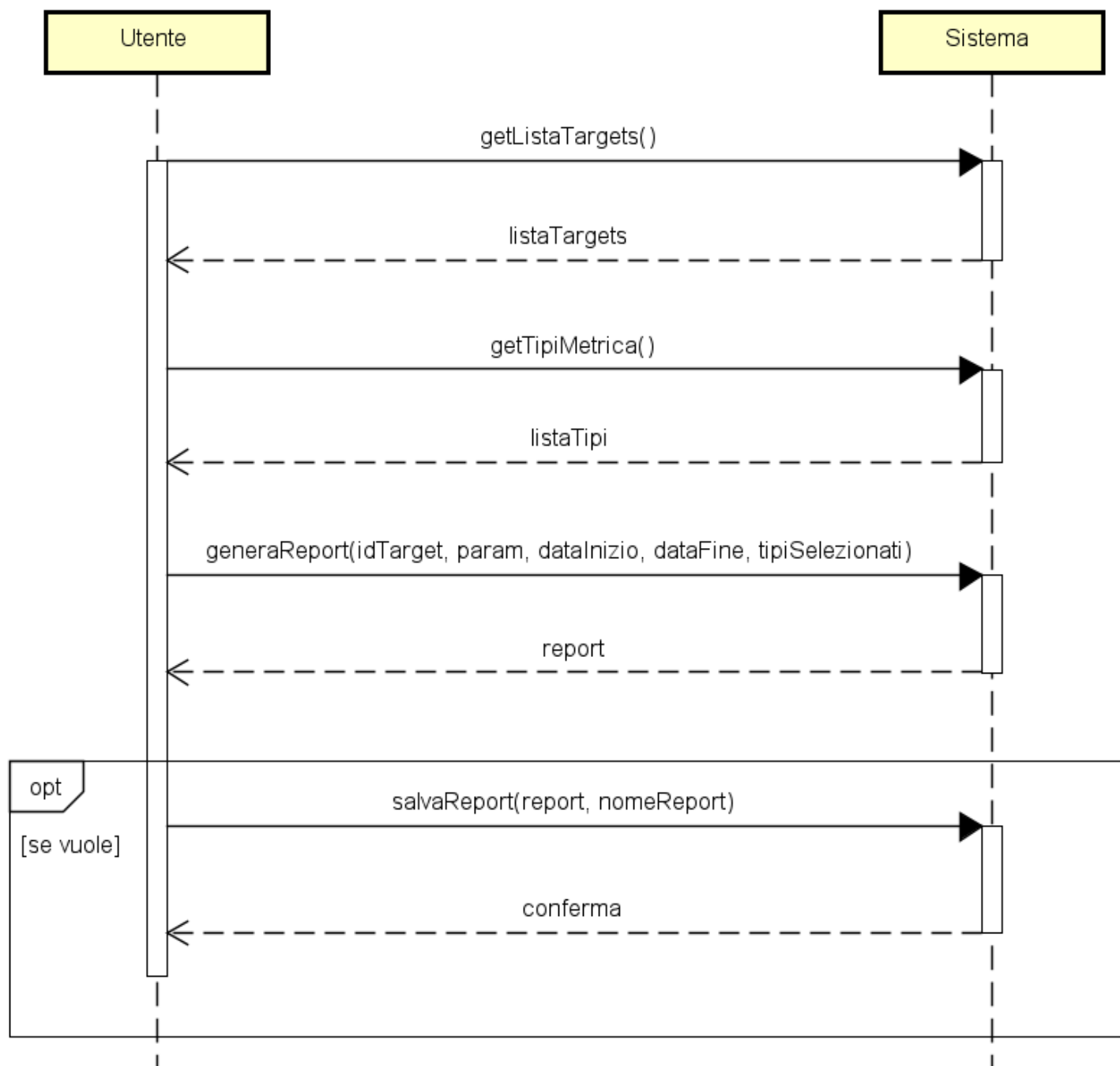


Diagramma di sequenza di Sistema



Contratti

Contratto CO1: getListaTargets

Operazione: `getListaTargets()`

Riferimenti: UC1.3: Consulta Metriche, UC6a: Crea Routine

Pre-condizioni: Il sistema è attivo. Esiste almeno un'istanza di `Target` registrata nel sistema.

Post-condizioni: Nessuna modifica allo stato del dominio.

Contratto CO2: getTipiMetrica

Operazione: `getTipiMetrica()`

Riferimenti: UC1.3: Consulta Metriche

Pre-condizioni: Il sistema è attivo.

Post-condizioni: Nessuna modifica allo stato del dominio.

Contratto CO3: generaReport

Operazione: generaReport(idTarget, dataInizio, dataFine, tipiSelezionati)

Riferimenti: UC1.3: Consulta Metriche

Pre-condizioni: Esiste nel sistema un'istanza *t* di Target con id corrispondente a idTarget. La dataInizio è antecedente o uguale alla dataFine. La Cronologia non è vuota.

Post-condizioni:

- È stata creata una nuova istanza *r* di Report.
- È stata creata una nuova istanza *f* di FiltroTemporale i cui attributi dataInizio e dataFine sono stati inizializzati con i parametri ricevuti.
- È stata formata un'associazione tra l'istanza *r* di Report e l'istanza *f* di FiltroTemporale.
- È stata formata un'associazione tra l'istanza *r* di Report e l'istanza *t* di Target.
- **Per ogni stringa presente in tipiSelezionati:**
 - È stata individuata l'istanza *tm* di TipoMetrica preesistente nel sistema corrispondente alla stringa corrente.
 - È stata creata un'istanza *m* di Metrica.
 - È stata formata un'associazione tra l'istanza *m* di Metrica e l'istanza *tm* di TipoMetrica.
 - Gli attributi valore e unitaMisura dell'istanza *m* sono stati valorizzati dal sistema elaborando la Cronologia.
 - È stata formata un'associazione tra l'istanza *r* di Report e l'istanza *m* di Metrica.

Contratto CO4: salvaReport

Operazione: salvaReport(report, nomeReport)

Riferimenti: UC1.3: Consulta Metriche

Pre-condizioni: Esiste un'istanza *r* di Report appena generata nel sistema (creata al passo precedente). Esiste un'istanza *a* di ArchivioReports gestita dallo SmartHub.

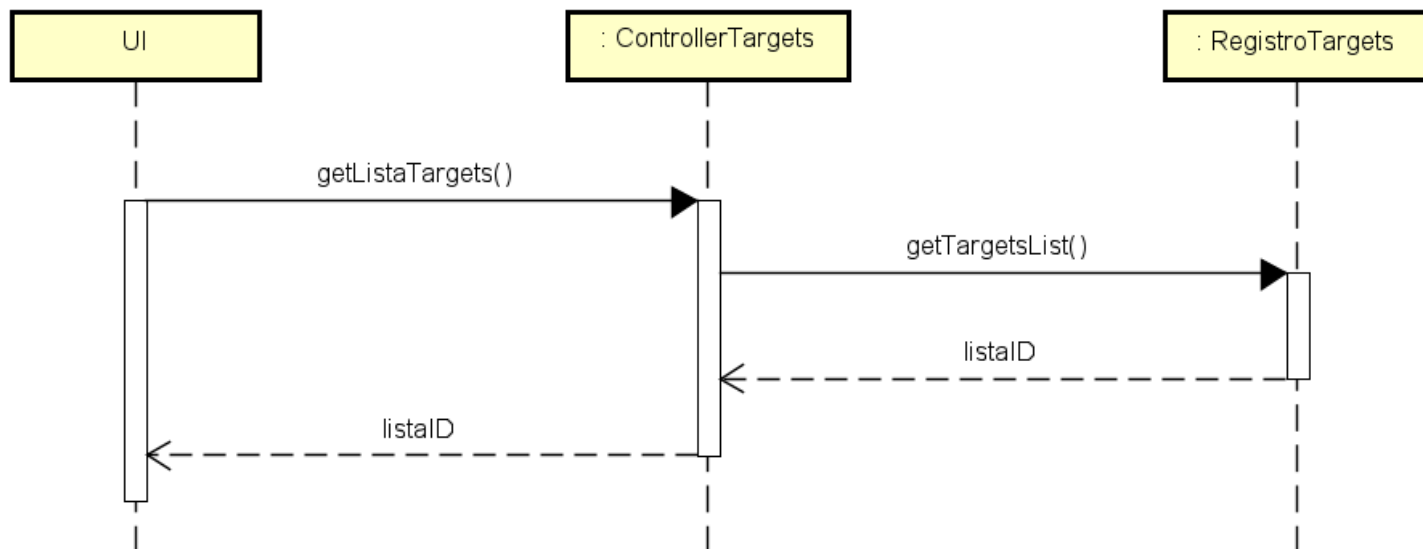
Post-condizioni:

- È stata formata un'associazione tra l'istanza *a* di ArchivioReports e l'istanza *r* di Report.

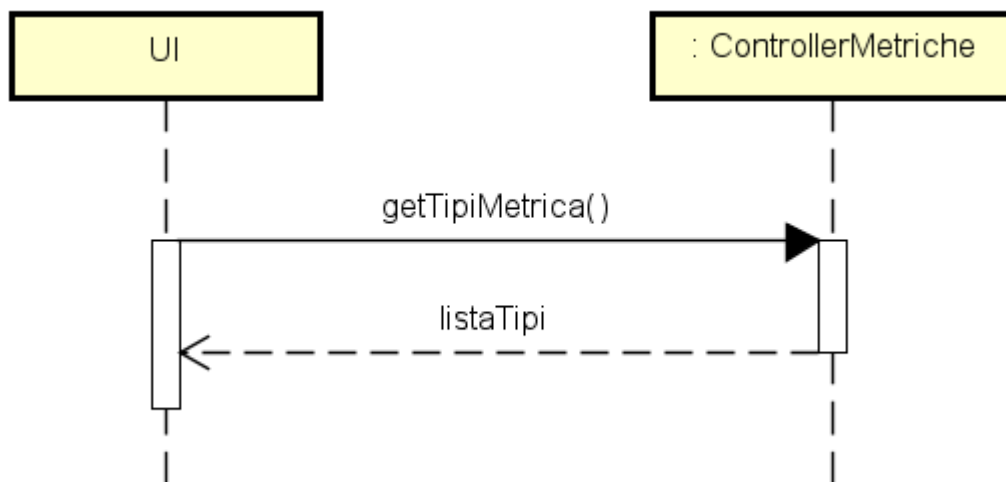
Progettazione

Diagrammi di sequenza

Op1: getListaTargets()

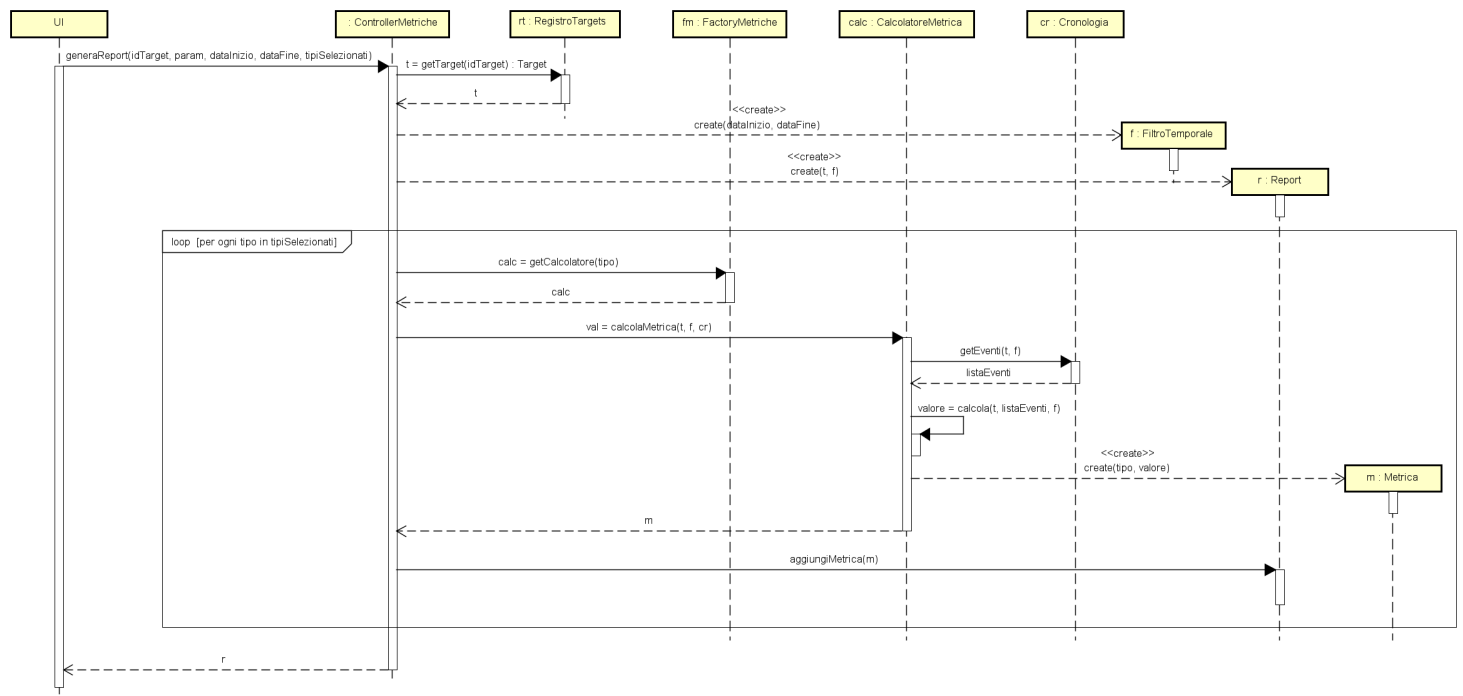


Op2: getTipiMetrica()



Op3: generaReport()

In base ai tipi di metrica selezionati dall'utente, tramite il pattern **SimpleFactory**, vengono istanziati dinamicamente i calcolatori opportuni. Ciascuno si occuperà di calcolare una specifica metrica.



Op4: salvaReport()

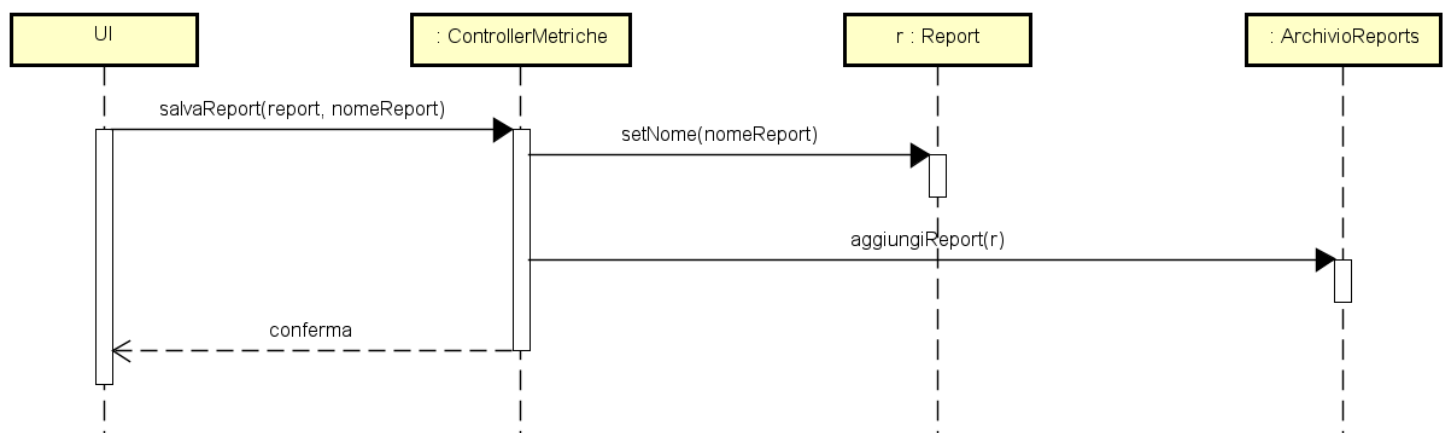
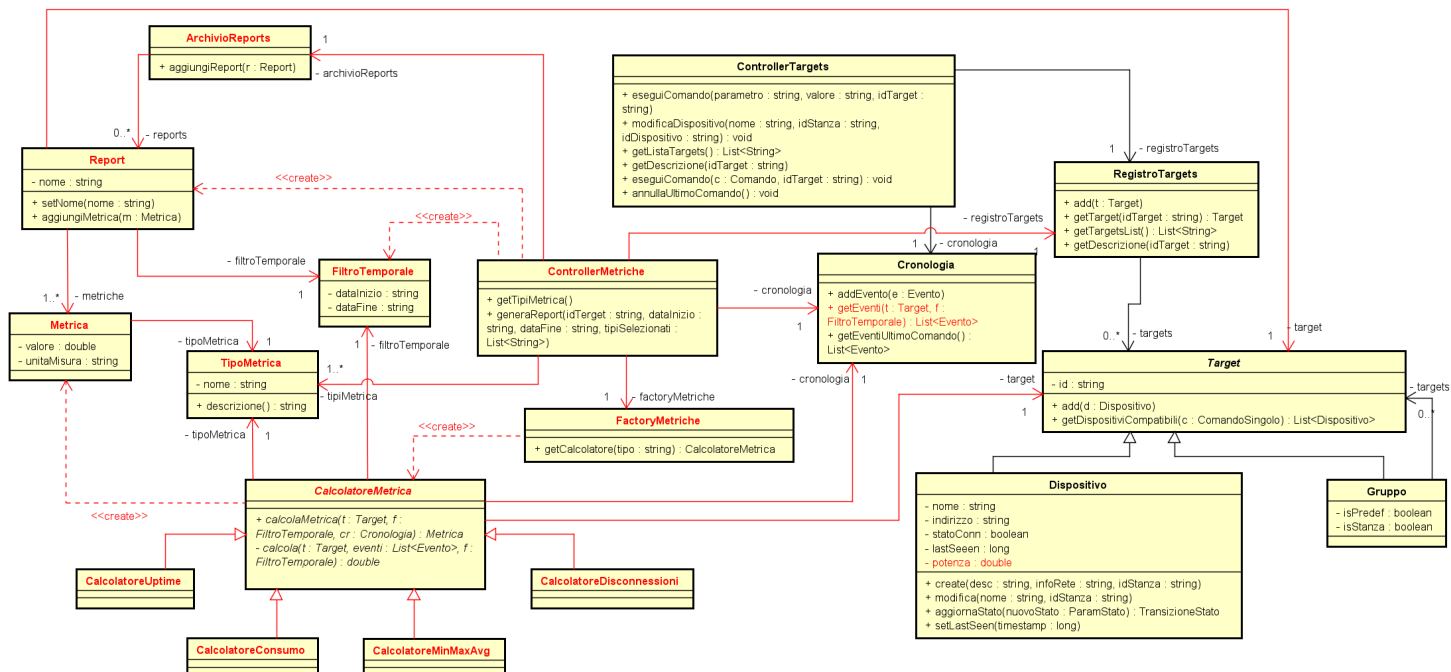


Diagramma delle classi



UC1.3: Consulta Metriche

Implementazione

Le seguenti classi sono state implementate:

- TipoMetrica
- FiltroTemporale
- Metrica
- Report
- ArchivioReports
- CalcolatoreMetrica
- CalcolatoreConsumo
- FactoryMetriche
- ControllerMetriche

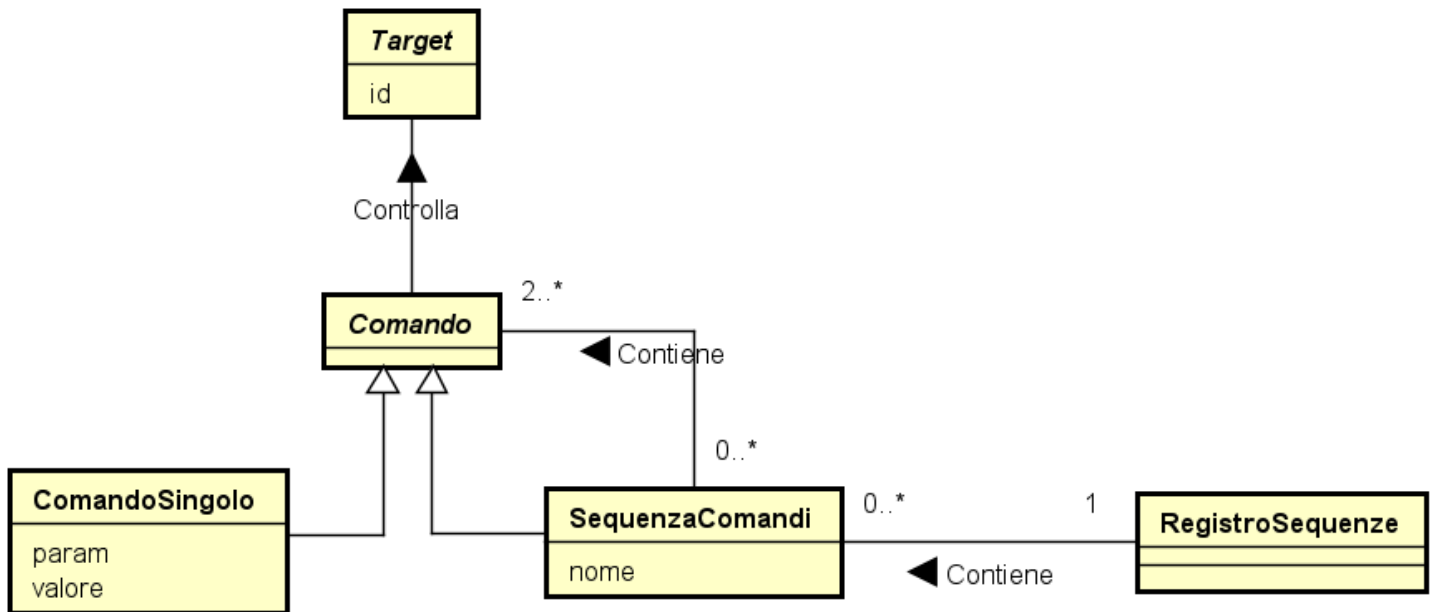
Inoltre per supportare il caso d'uso sono stati aggiunti l'attributo `potenza` a `Dispositivo` ed il metodo `getEventi(target, filtroTemporale)` a `Cronologia`.

UC8: Esegui sequenza di comandi

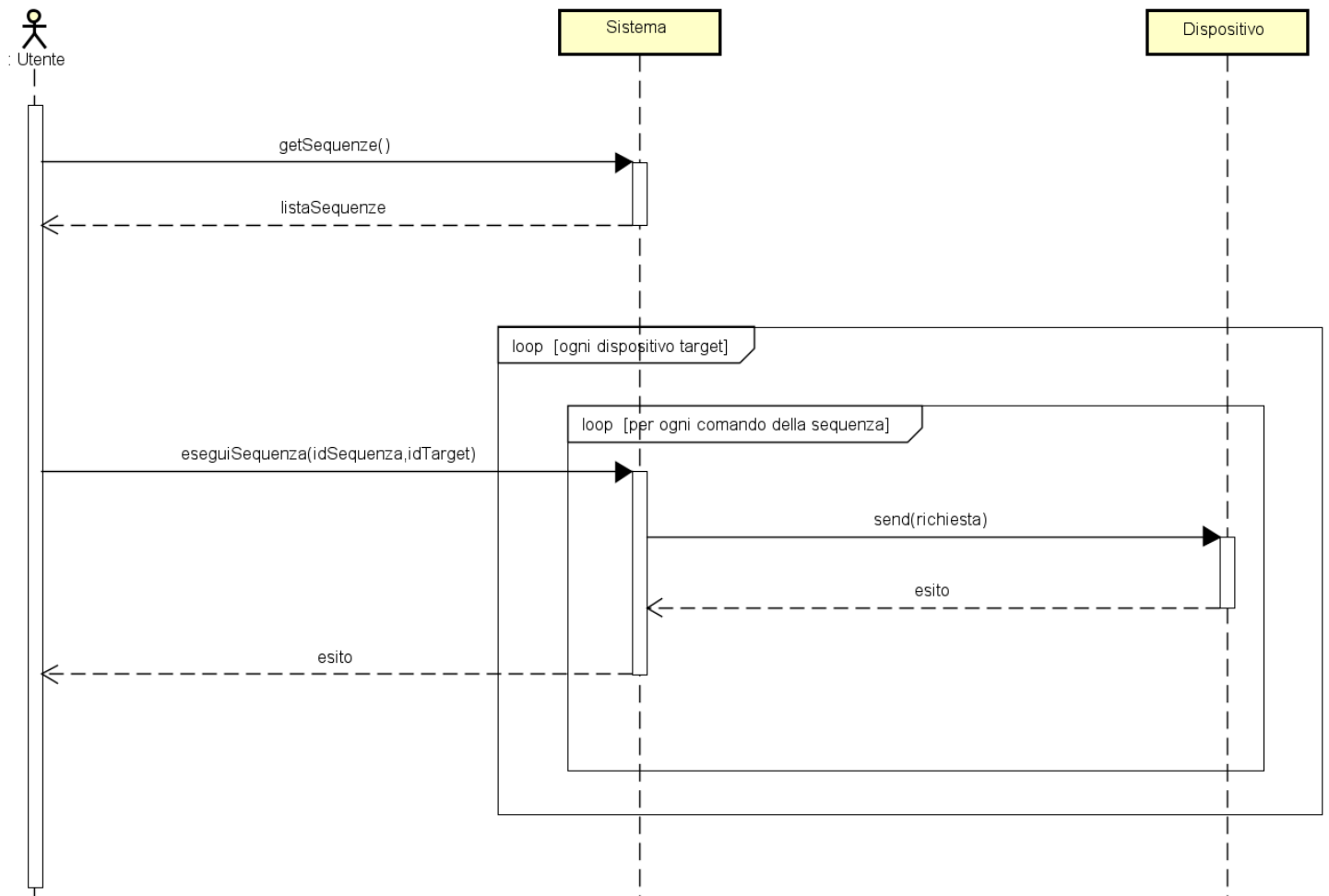
Per l'implementazione delle sequenze di comandi introdurremo un nuovo composite ed integreremo il flusso a quello del caso d'uso UC2: Comanda Dispositivi.

Analisi

Modello Dominio



SSD



Contratti

Contratto CO1: getSequenze

- **Operazione:** `getSequenze()`
- **Riferimenti:** UC8: Esegui Sequenza
- **Pre-condizioni:** Il sistema è inizializzato e il `RegistroSequenze` è stato caricato.
- **Post-condizioni:** Lo stato del sistema non è cambiato.

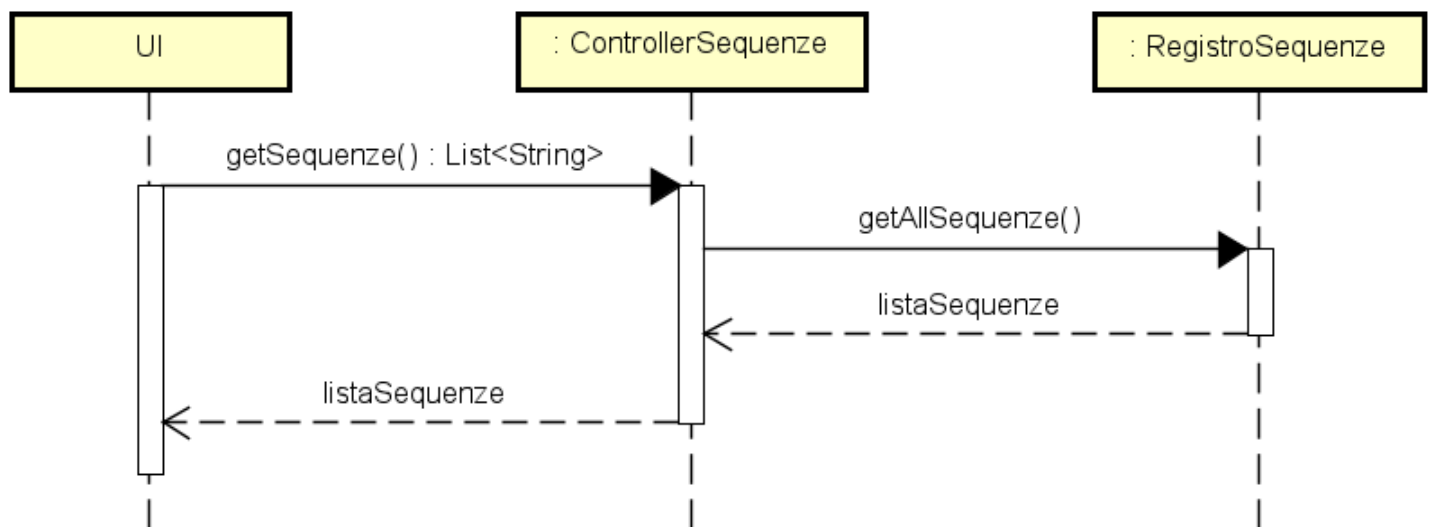
Contratto CO2: eseguiSequenza

- **Operazione:** `eseguiSequenza(idSequenza, idTarget)`
- **Riferimenti:** UC8: Esegui Sequenza
- **Pre-condizioni:** Esiste un'istanza `t` di `Target` corrispondente a `idTarget` nel `RegistroTargets`.
 - Esiste un'istanza `s` di `SequenzaComandi` corrispondente a `idSequenza` nel `RegistroSequenze`.
- **Post-condizioni:**
 - Lo stato operativo delle istanze di `Dispositivo` compatibili contenute in `t` è stato aggiornato.

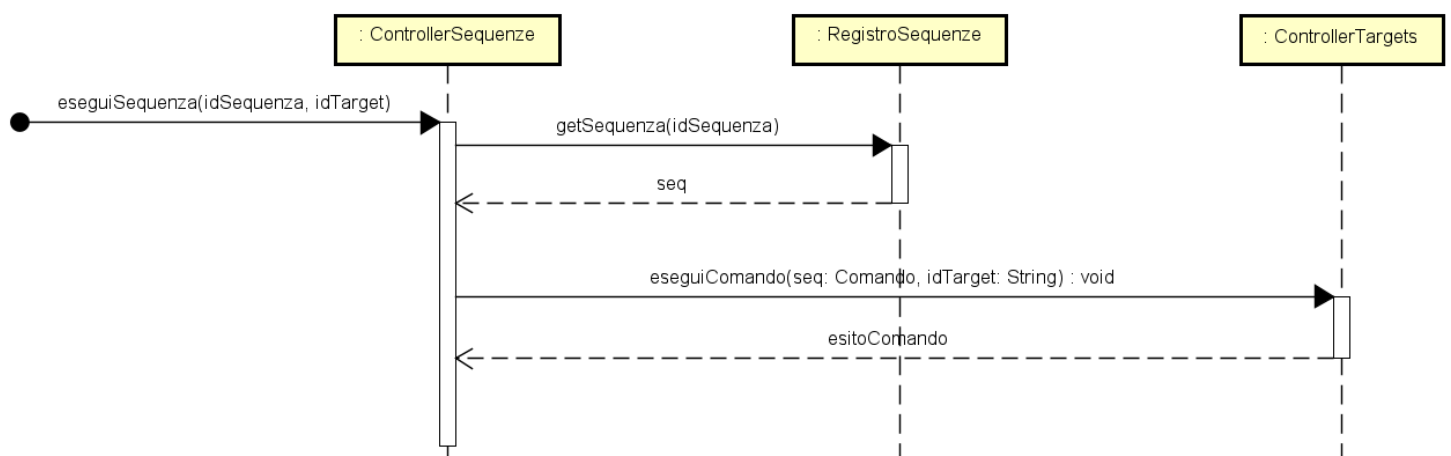
Progettazione

Diagrammi di sequenza

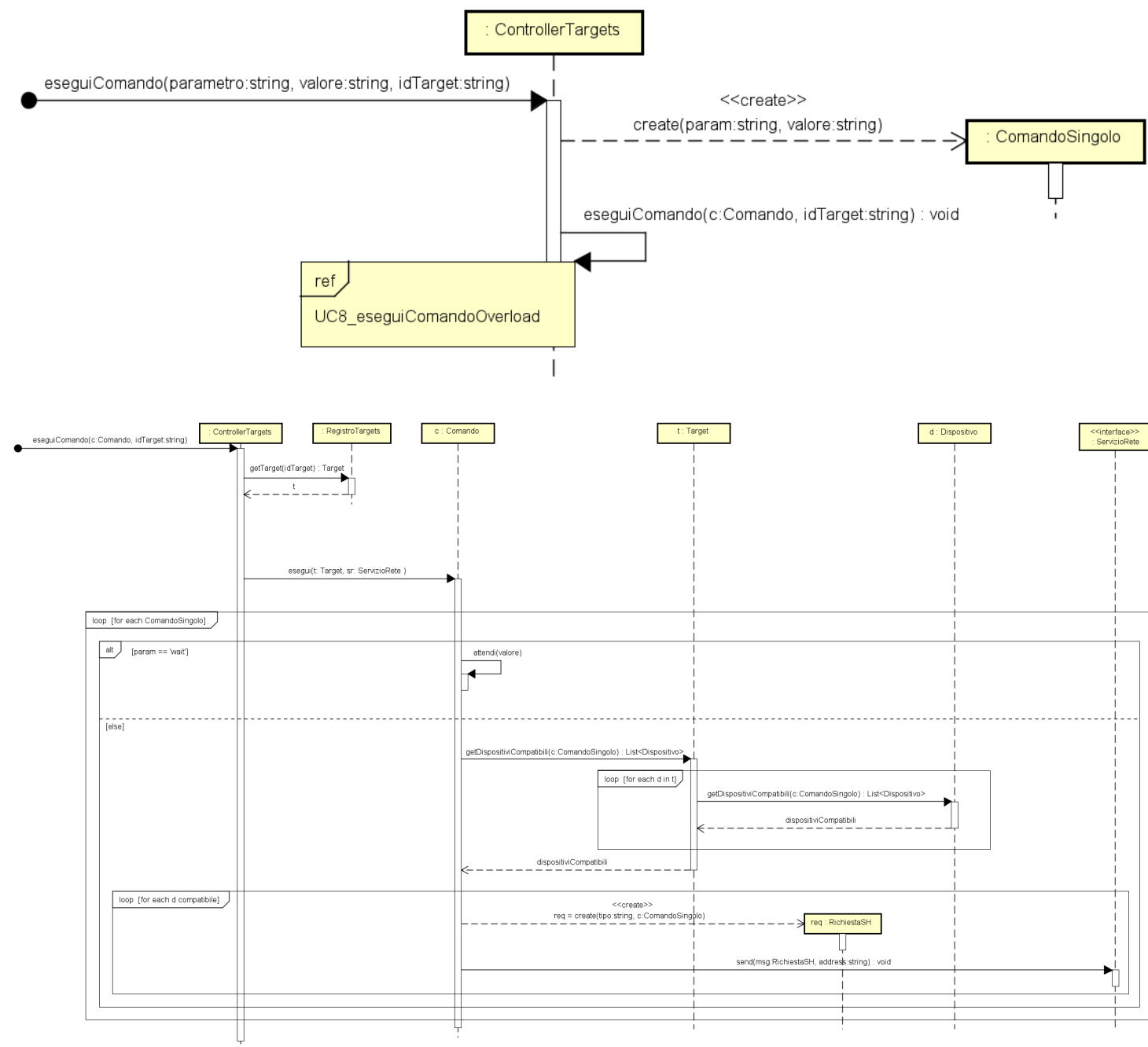
Op1: getSequenze



Op2: eseguiSequenza

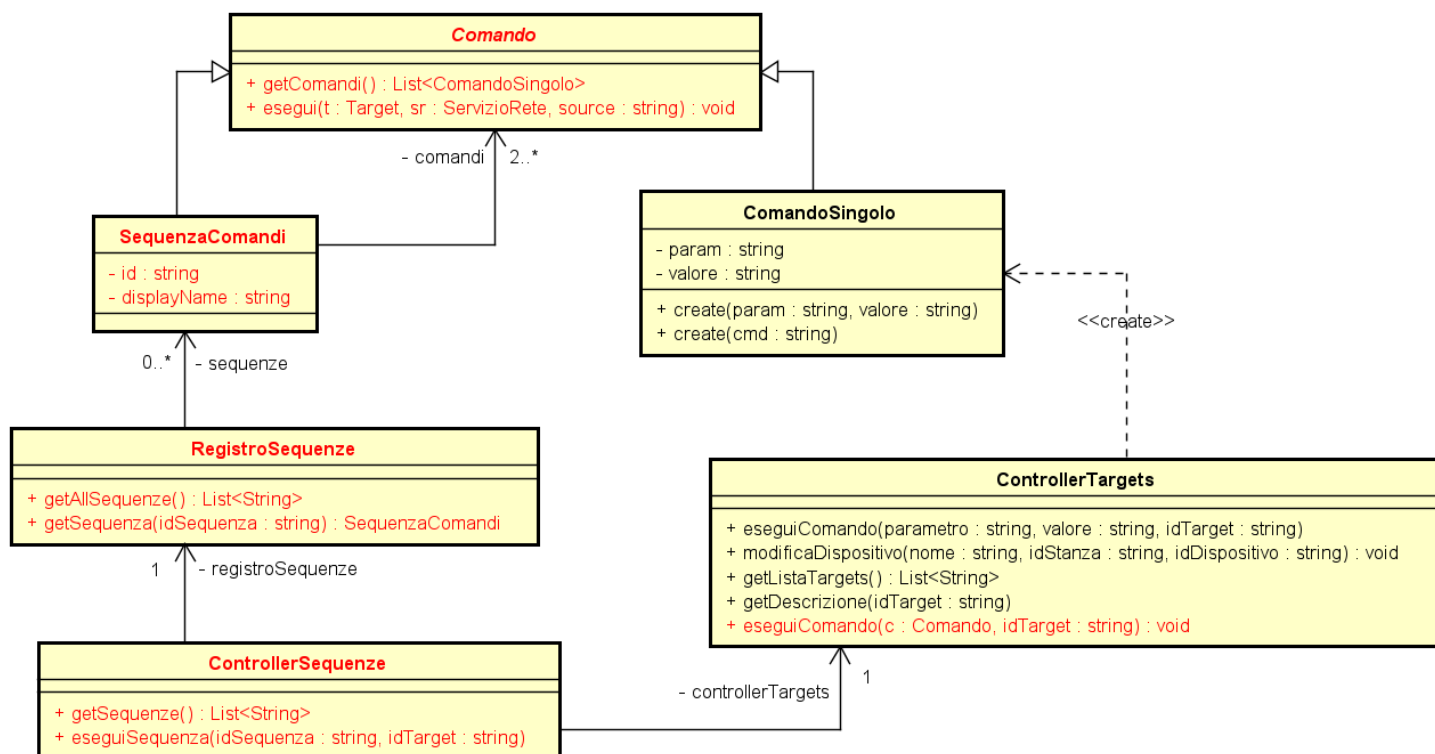


Come si vede dal diagramma, eseguiSequenza include un overload dell'operazione eseguiComando() progettata per il caso d'uso UC2: Comanda Dispositivo. Al fine di integrare l'esecuzione delle sequenze al flusso preesistente, l'operazione eseguiComando() è stata riprogettata come segue:



In particolare abbiamo applicato il pattern **Command** delegando al comando la sua esecuzione al fine di aumentare la coesione, supportare il polimorfismo e allo stesso tempo spostare la logica di business via dal controller.

Diagramma delle classi



Implementazione

Classi implementate:

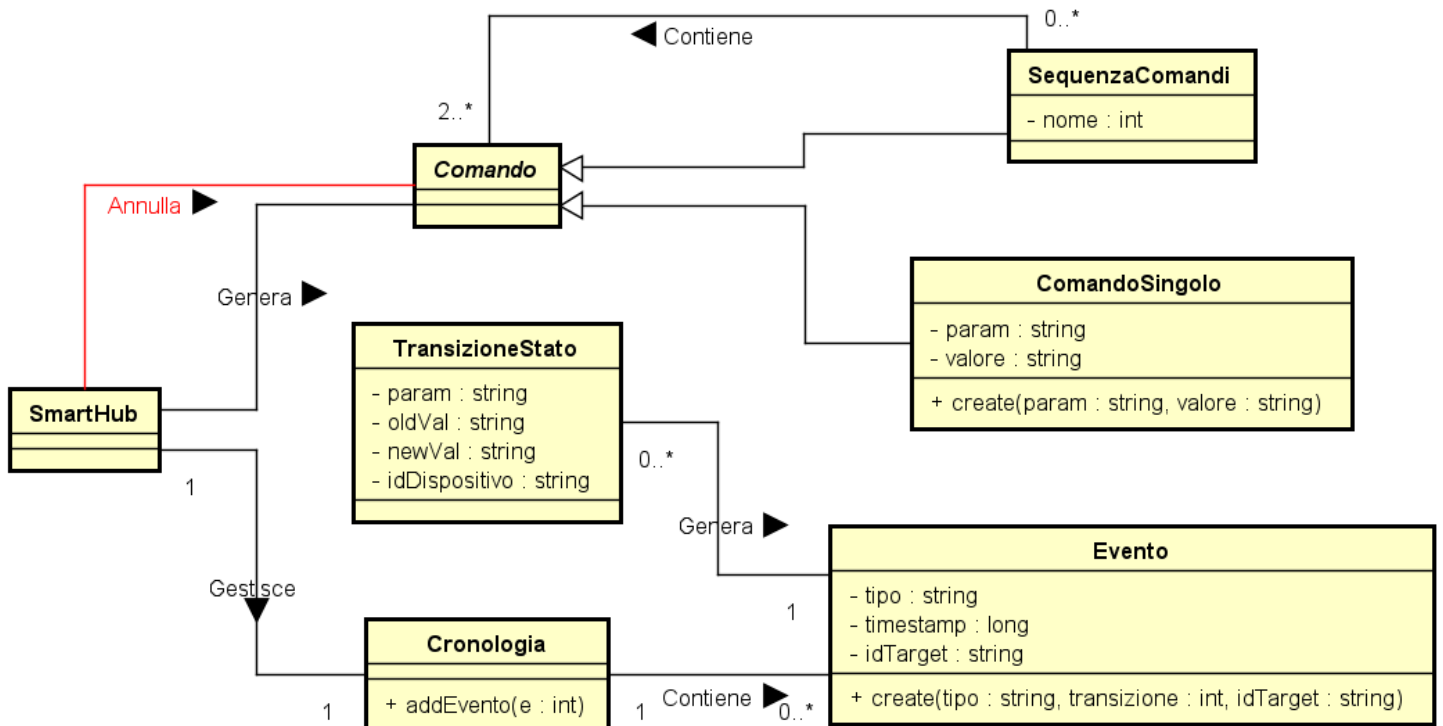
- Comando
- ComandoSingolo
- SequenzaComandi
- RegistroSequenze
- ControllerSequenze
- Modificato il metodo `eseguiComando()` di **ControllerTargets** come specificato.

UC5.1: Gestisci Cronologia - Annulla ultimo comando

Analisi

Modello di Dominio

Il dominio rimane pressoché invariato.



SSD



Contratti

Contratto CO3: annullaUltimoComando

Operazione: `annullaUltimoComando()`

Riferimenti: UC5.1: Annulla Ultima Comando

Pre-condizioni: Esiste almeno un `Evento` e di tipo 'userCmd' nella `Cronologia`.

Post-condizioni:

- Lo stato operativo delle istanze di `Dispositivo` coinvolte nell'ultimo comando registrato è stato ripristinato al loro valore precedente il comando.
- Le istanze di `Evento` relative ai comandi di compensazione inviati sono state registrate nella `Cronologia`.

Progettazione

Diagrammi di sequenza

Si è deciso di inviare degli eventi di compensazione per evitare di falsare lo storico del funzionamento dei dispositivi.

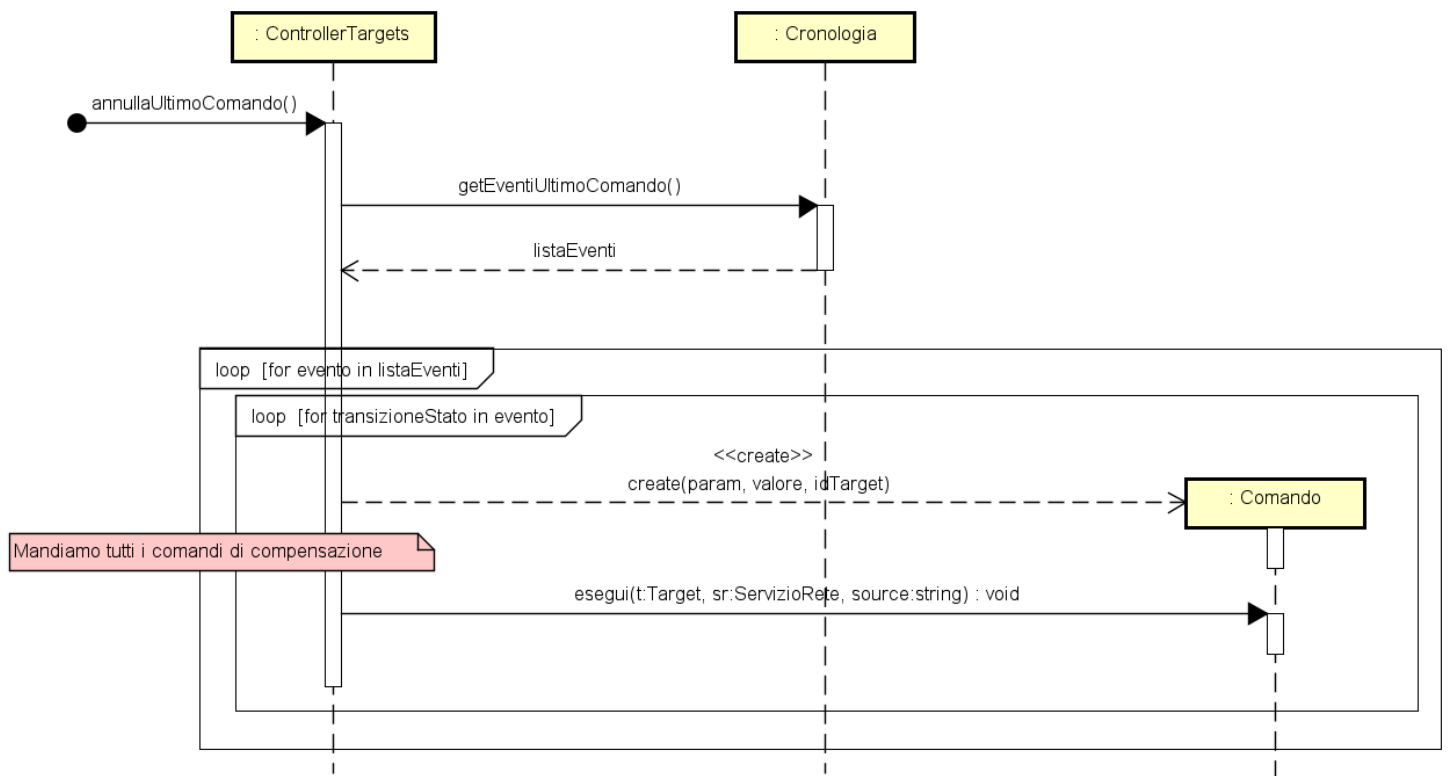
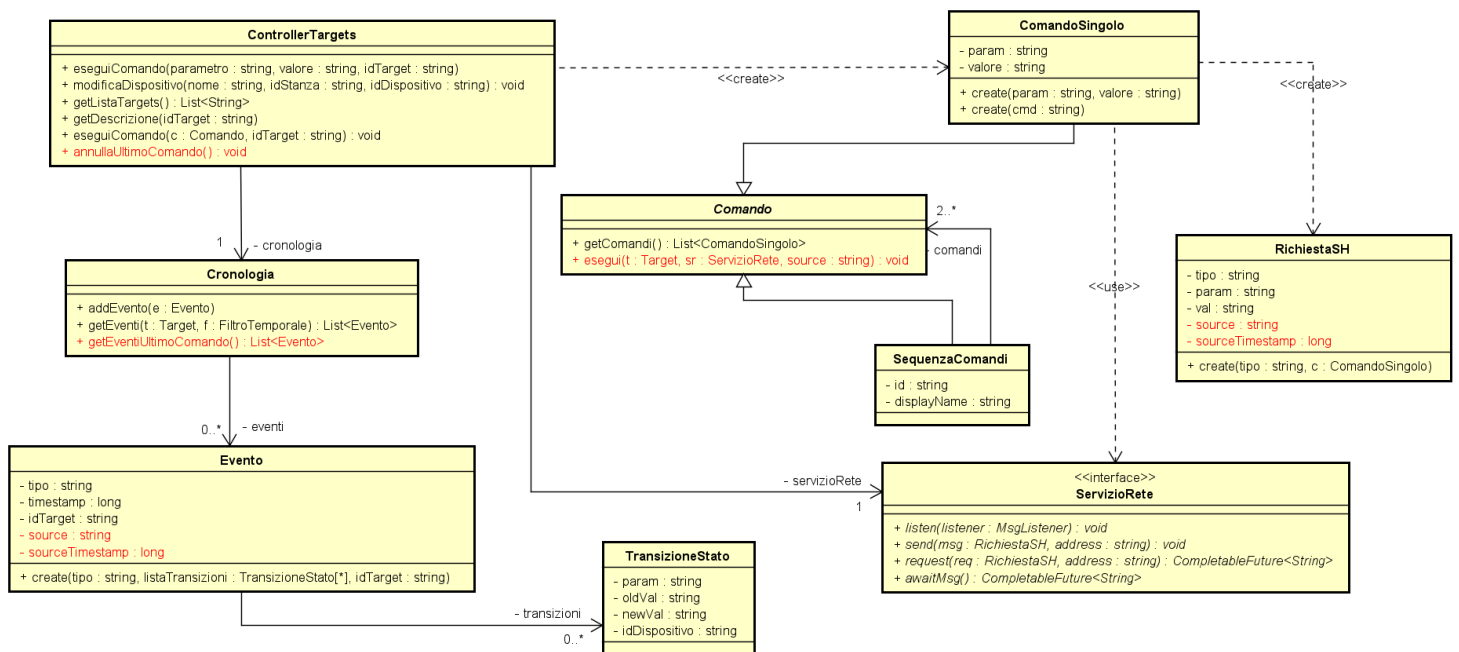


Diagramma delle Classi



Implementazione

Sono stati introdotti i seguenti attributi per supportare la ricerca degli eventi relativi all'ultimo comando:

- `sourceTimestamp` : istante di tempo corrispondente alla chiamata del metodo per l'esecuzione del comando.
 - serve a ricondurre gli eventi alla stessa chiamata del metodo e individuare dunque tutti quelli relativi all'ultimo comando
 - utile per evitare di scorrere tutta la lista nella ricerca (nessun evento avrà `timestamp` minore rispetto al `sourceTimestamp` della chiamata che lo ha generato.)
 - `source` : serve a distinguere comandi impartiti dall'utente da quelli lanciati da una `Routine` o dal sistema
- Il metodo `annullaUltimoComando()` è stato implementato come specificato.

Refactor

- L'attributo `sourceTarget` è stato incluso all'interno di `RichiestaSH` per garantire un tracciamento rigoroso dell'origine dei comandi.
- A livello di rete, il meccanismo di keep-alive previsto dal UC7: Monitoraggio è stato ottimizzato introducendo ping per la prima sincronizzazione (PING_SYNC), senza i quali il sistema non era in grado di rilevare lo stato dei dispositivi alla connessione.
 - Il `ControllerMonitoraggio` è stato aggiornato per gestire i nuovi PING_SYNC.